

```

        {
            point.x + m_offset.x,
            point.y + m_offset.y,
            0.0f
        });
    }
}

void AddVisual(winrt::float2 const point)
{
    winrt::Compositor compositor = m_visuals.Compositor();
    winrt::SpriteVisual visual = compositor.CreateSpriteVisual();

    static winrt::Color colors[] =
    {
        { 0xDC, 0x5B, 0x9B, 0xD5 },
        { 0xDC, 0xED, 0x7D, 0x31 },
        { 0xDC, 0x70, 0xAD, 0x47 },
        { 0xDC, 0xFF, 0xC0, 0x00 }
    };

    static unsigned last = 0;
    unsigned const next = ++last % _countof(colors);
    visual.Brush(compositor.CreateColorBrush(colors[next]));

    float const BlockSize = 100.0f;

    visual.Size(
    {
        BlockSize,
        BlockSize
    });

    visual.Offset(
    {
        point.x - BlockSize / 2.0f,
        point.y - BlockSize / 2.0f,
        0.0f,
    });

    m_visuals.InsertAtTop(visual);

    m_selected = visual;
    m_offset.x = -BlockSize / 2.0f;
    m_offset.y = -BlockSize / 2.0f;
}
};

int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int)
{
    winrt::init_apartment();

    winrt::Uri uri(L"http://aka.ms/cppwinrt");
    std::wstringstream wstringstream;
    wstringstream << L"C++/WinRT: " << uri.Domain().c_str() << std::endl;

```

```

// Convert from a C++/WinRT type to a C++/CX type.
cx::Uri^ cx = to_cx<cx::Uri>(uri);
wstringstream << L"C++/CX: " << cx->Domain->Data() << std::endl;
::OutputDebugString(wstringstream.str().c_str());

// Convert from a C++/CX type to a C++/WinRT type.
winrt::Uri uri_from_cx = from_cx<winrt::Uri>(cx);
WINRT_ASSERT(uri.Domain() == uri_from_cx.Domain());
WINRT_ASSERT(uri == uri_from_cx);

winrt::CoreApplication::Run(winrt::make<App>());
}

```

Important APIs

- [IUnknown interface](#)
- [QueryInterface function](#)
- [winrt::get_abi function](#)
- [winrt::put_abi function](#)

Related topics

- [C++/CX](#)
- [Move to C++/WinRT from C++/CX](#)
- [Asynchrony, and interop between C++/WinRT and C++/CX](#)

Asynchrony, and interop between C++/WinRT and C++/CX

Article • 10/20/2022

Tip

Although we recommend that you read this topic from the beginning, you can jump straight to a summary of interop techniques in the [Overview of porting C++/CX async to C++/WinRT](#) section.

This is an advanced topic related to gradually porting to [C++/WinRT](#) from [C++/CX](#). This topic picks up where the topic [Interop between C++/WinRT and C++/CX](#) leaves off.

If the size or complexity of your codebase makes it necessary to port your project gradually, then you'll need a porting process in which for a time C++/CX and C++/WinRT code exists side by side in the same project. If you have asynchronous code, then you might need to have Parallel Patterns Library (PPL) task chains and coroutines exist side by side in your project as you gradually port your source code. This topic focuses on techniques for interoperating between asynchronous C++/CX code and asynchronous C++/WinRT code. You can use these techniques individually, or together. The techniques allow you to make gradual, controlled, local changes along the path toward porting your entire project, without having each change cascade uncontrollably throughout the project.

Before reading this topic, it's a good idea to read [Interop between C++/WinRT and C++/CX](#). That topic shows you how to prepare your project for gradual porting. It also introduces two helper functions that you can use to convert a C++/CX object into a C++/WinRT object (and vice versa). This topic about asynchrony builds on that info, and it uses those helper functions.

Note

There are some limitations to porting gradually from C++/CX to C++/WinRT. If you have a **Windows Runtime component** project, then porting gradually is not possible, and you'll need to port the project in one pass. And for a XAML project, at any given time your XAML page types must be *either* all C++/WinRT *or* all C++/CX. For more info, see the topic [Move to C++/WinRT from C++/CX](#).

The reason an entire topic is dedicated to asynchronous code interop

Porting from C++/CX to C++/WinRT is generally straightforward, with the one exception of moving from [Parallel Patterns Library \(PPL\)](#) tasks to coroutines. The models are different. There isn't a natural one-to-one mapping from PPL tasks to coroutines, and there's no simple way (that works for all cases) to mechanically port the code.

The good news is that conversion from tasks to coroutines results in significant simplifications. And development teams routinely report that once they're over the hurdle of porting their asynchronous code, the remainder of the porting work is largely mechanical.

Often, an algorithm was originally written to suit synchronous APIs. And then that was translated into tasks and explicit continuations—the result often being an inadvertent obfuscation of the underlying logic. For example, loops become recursion; if-else branches turn into a nested tree (a chain) of tasks; shared variables become `shared_ptr`. To deconstruct the often unnatural structure of PPL source code, we recommend that you first step back and understand the intent of the original code (that is, discover the original synchronous version). And then insert `co_await` (cooperatively await) into the appropriate places.

For that reason, if you have a C# (rather than C++/CX) version of the asynchronous code from which to begin your port, then that can give you an easier time, and a cleaner port. C# code uses `await`. So C# code already essentially follows a philosophy of beginning with a synchronous version and then inserting `await` into the appropriate places.

If you *don't* have a C# version of your project, then you can use the techniques described in this topic. And once you've ported to C++/WinRT, the structure of your async code will then be easier to port to C#, should you wish to.

Some background in asynchronous programming

So that we have a common frame of reference for asynchronous programming concepts and terminology, let's briefly set the scene regarding Windows Runtime asynchronous programming in general, and also how the two C++ language projections are each, in their different ways, layered on top of that.

Your project has methods that do work asynchronously, and there are two main kinds.

- It's common to want to wait on the completion of asynchronous work before you do something else. A method that returns an asynchronous operation object is one that you can wait on.
- But sometimes you don't want or need to wait on the completion of work done asynchronously. In that case it's more efficient for the asynchronous method *not* to return an asynchronous operation object. An asynchronous method such as that—one that you don't wait on—is known as a *fire-and-forget* method.

Windows Runtime async objects (IAsyncXxx)

The **Windows::Foundation** Windows Runtime namespace contains four types of asynchronous operation object.

- [IAsyncAction](#),
- [IAsyncActionWithProgress<TProgress>](#),
- [IAsyncOperation<TResult>](#), and
- [IAsyncOperationWithProgress<TResult, TProgress>](#).

In this topic, when we use the convenient shorthand of **IAsyncXxx**, we're referring either to these types collectively; or we're talking about one of the four types without needing to specify which one.

C++/CX async

Asynchronous C++/CX code makes use of [Parallel Patterns Library \(PPL\)](#) tasks. A PPL task is represented by the [concurrency::task](#) class.

Typically, an asynchronous C++/CX method chains PPL tasks together by using lambda functions with [concurrency::create_task](#) and [concurrency::task::then](#). Each lambda function returns a task which, when it completes, produces a value that is then passed into the lambda of the task's *continuation*.

Alternatively, instead of calling `create_task` to create a task, an asynchronous C++/CX method can call [concurrency::create_async](#) to create an **IAsyncXxx**^.

So the return type of an asynchronous C++/CX method can be a PPL task, or an **IAsyncXxx**^.

In either case, the method itself uses the `return` keyword to return an asynchronous object which, when it completes, produces the value that the caller actually wants (perhaps a file, an array of bytes, or a Boolean).

ⓘ Note

If an asynchronous C++/CX method returns an **IAsyncXxx**^, then the **TResult** (if any) is limited to being a Windows Runtime type. A Boolean value, for example, is a Windows Runtime type; but a C++/CX projected type (for example, **Platform::Array<byte>**^) is not.

C++/WinRT async

C++/WinRT integrates C++ coroutines into the programming model. Coroutines and the `co_await` statement provide a natural way to cooperatively wait for a result.

Each of the **IAsyncXxx** types is projected into a corresponding type in the **winrt::Windows::Foundation** C++/WinRT namespace. Let's refer to those as **winrt::IAsyncXxx** (as compared to the **IAsyncXxx**^ of C++/CX).

The return type of a C++/WinRT coroutine is either a **winrt::IAsyncXxx**, or **winrt::fire_and_forget**. And instead of using the `return` keyword to return an asynchronous object, a coroutine uses the `co_return` keyword to cooperatively return the value that the caller actually wants (perhaps a file, an array of bytes, or a Boolean).

If a method contains at least one `co_await` statement (or at least one `co_return` or `co_yield`), then the method is a coroutine for that reason.

For more info, and code examples, see [Concurrency and asynchronous operations with C++/WinRT](#).

The Direct3D game sample (Simple3DGameDX)

This topic contains walkthroughs of several specific programming techniques that illustrate how to gradually port asynchronous code. To serve as a case study, we'll be using the C++/CX version of the [Direct3D game sample](#) (which is called **Simple3DGameDX**). We'll show some examples of how you can take the original C++/CX source code in that project and gradually port its asynchronous code to C++/WinRT.

- Download the ZIP from the link above, and unzip it.
- Open the C++/CX project (it's in the folder named `cpp`) in Visual Studio.
- You'll then need to add C++/WinRT support to the project. The steps that you follow to do that are described in [Taking a C++/CX project and adding C++/WinRT](#)

[support](#). In that section, the step about adding the `interop_helpers.h` header file to your project is particularly important because we'll be depending on those helper functions in this topic.

- Finally, add `#include <pplawait.h>` to `pch.h`. That gives you coroutine support for PPL (there's more about that support in the following section).

Don't build yet, otherwise you'll get errors about `byte` being ambiguous. Here's how to resolve that.

- Open `BasicLoader.cpp`, and comment out `using namespace std;`.
- In that same source code file, you'll then need to qualify `shared_ptr` as `std::shared_ptr`. You can do that with a search-and-replace within that file.
- Then qualify `vector` as `std::vector`, and `string` as `std::string`.

The project now builds again, has C++/WinRT support, and contains the `from_cx` and `to_cx` interop helper functions.

You now have the `Simple3DGameDX` project ready to follow along with the code walkthroughs in this topic.

Overview of porting C++/CX async to C++/WinRT

In a nutshell, as we port, we'll be changing PPL task chains into calls to `co_await`. We'll be changing the return value of a method from a PPL task to a C++/WinRT `wintrt::IAsyncXxx` object. And we'll also be changing any `IAsyncXxx^` to a C++/WinRT `wintrt::IAsyncXxx`.

You'll recall that a coroutine is any method that calls `co_xxx`. A C++/WinRT coroutine uses `co_return` to cooperatively return its value. Thanks to the coroutine support for PPL (courtesy of `pplawait.h`), you can also use `co_return` to return a PPL task from a coroutine. And you can also `co_await` both tasks and `IAsyncXxx`. But you can't use `co_return` to return an `IAsyncXxx^`. The table below describes support for interop between the various asynchronous techniques with `pplawait.h` in the picture.

Method	Can you <code>co_await</code> it?	Can you <code>co_return</code> from it?
Method returns <code>task<void></code>	Yes	Yes
Method returns <code>task<T></code>	No	Yes

Method	Can you <code>co_await</code> it?	Can you <code>co_return</code> from it?
Method returns <code>IAsyncXxx^</code>	Yes	No. But you wrap <code>create_async</code> around a task that uses <code>co_return</code> .
Method returns <code>winrt::IAsyncXxx</code>	Yes	Yes

Use this next table to jump straight to the section in this topic that describes an interop technique of interest, or just continue reading from here.

Async interop technique	Section in this topic
Use <code>co_await</code> to await a <code>task<void></code> method from within a fire-and-forget method, or within a constructor.	Await <code>task<void></code> within a fire-and-forget method
Use <code>co_await</code> to await a <code>task<void></code> method from within a <code>task<void></code> method.	Await <code>task<void></code> within a <code>task<void></code> method
Use <code>co_await</code> to await a <code>task<void></code> method from within a <code>task<T></code> method.	Await <code>task<void></code> within a <code>task<T></code> method
Use <code>co_await</code> to await an <code>IAsyncXxx^</code> method.	Await an <code>IAsyncXxx^</code> in a <code>task</code> method, leaving the rest of the project unchanged
Use <code>co_return</code> within a <code>task<void></code> method.	Await <code>task<void></code> within a <code>task<void></code> method
Use <code>co_return</code> within a <code>task<T></code> method.	Await an <code>IAsyncXxx^</code> in a <code>task</code> method, leaving the rest of the project unchanged
Wrap <code>create_async</code> around a task that uses <code>co_return</code> .	Wrap <code>create_async</code> around a task that uses <code>co_return</code>
Port <code>concurrency::wait</code> .	Port <code>concurrency::wait</code> to <code>co_await winrt::resume_after</code>
Return <code>winrt::IAsyncXxx</code> instead of <code>task<void></code> .	Port a <code>task<void></code> return type to <code>winrt::IAsyncXxx</code>
Convert a <code>winrt::IAsyncXxx<T></code> (T is primitive) to a <code>task<T></code> .	Convert a <code>winrt::IAsyncXxx<T></code> (T is primitive) to a <code>task<T></code>
Convert a <code>winrt::IAsyncXxx<T></code> (T is a Windows Runtime type) to a <code>task<T^></code> .	Convert a <code>winrt::IAsyncXxx<T></code> (T is a Windows Runtime type) to a <code>task<T^></code>

And here's a short code example illustrating some of the support.

C++/CX

```
#include <ppltasks.h>
#include <pplawait.h>
#include <winrt/Windows.Foundation.h>

concurrency::task<bool> TaskAsync()
{
    co_return true;
}

Windows::Foundation::IAsyncOperation<bool>^ IAsyncXxxCppCXAsync()
{
    // co_return true; // Error! Can't do that. But you can do
    // the following.
    return concurrency::create_async([=]() -> concurrency::task<bool> {
        co_return true;
    });
}

winrt::Windows::Foundation::IAsyncOperation<bool> IAsyncXxxCppWinRTAsync()
{
    co_return true;
}

concurrency::task<bool> CppCXAsync()
{
    bool b1 = co_await TaskAsync();
    bool b2 = co_await IAsyncXxxCppCXAsync();
    co_return co_await IAsyncXxxCppWinRTAsync();
}

winrt::fire_and_forget CppWinRTAsync()
{
    bool b1 = co_await TaskAsync();
    bool b2 = co_await IAsyncXxxCppCXAsync();
    bool b3 = co_await IAsyncXxxCppWinRTAsync();
}
```

❗ Important

Even with these great interop options, porting gradually depends on choosing changes that we can make surgically that don't affect the rest of the project. We want to avoid tugging at an arbitrary loose end, and thereby unraveling the structure of the whole project. For that, we have to do things in a particular order. Next we'll look closely at some examples of making these kinds of async-related porting/interop changes.

Await a `task<void>` method, leaving the rest of the project unchanged

A method that returns `task<void>` performs work asynchronously, and it returns an asynchronous operation object, but it doesn't ultimately produce a value. We can `co_await` a method like that.

So a good place to start porting async code gradually is to find places where you call such methods. Those places will involve creating and/or returning a task. They might also involve the kind of task chain where no value is passed from each task to its continuation. In places like that, you can just replace the async code with `co_await` statements, as we'll see.

❗ Note

As this topic progresses, you'll see the benefit to this strategy. Once a particular `task<void>` method is being called exclusively via `co_await`, you're then free to port that method to C++/WinRT, and have it return a `winrt::IAsyncXxx`.

Let's find some examples. Open the **Simple3DGameDX** project (see [The Direct3D game sample](#)).

📌 Important

In the examples that follow, as you see the implementations of methods being changed, bear in mind that we don't need to change the *callers* of the methods we're changing. These changes are localized, and they don't cascade through the project.

Await `task<void>` within a fire-and-forget method

Let's start with awaiting `task<void>` within *fire-and-forget* methods, since that's the simplest case. These are methods that do work asynchronously, but the caller of the method doesn't wait for that work to complete. You just call the method and forget it, despite the fact that it completes asynchronously.

Look toward the root of your project's dependency graph for `void` methods that contain `create_task` and/or task chains where only `task<void>` methods are called.

In `Simple3DGameDX`, you'll find code like that in the implementation of the method `GameMain::Update`. It's in the source code file `GameMain.cpp`.

GameMain::Update

Here's an extract from the C++/CX version of the method, showing the two parts of the method that complete asynchronously.

C++/CX

```
void GameMain::Update()
{
    ...
    case UpdateEngineState::WaitingForPress:
        ...
        m_game->LoadLevelAsync().then([this]()
        {
            m_game->FinalizeLoadLevel();
            m_updateState = UpdateEngineState::ResourcesLoaded;
        }, task_continuation_context::use_current());
        ...
    case UpdateEngineState::Dynamics:
        ...
        m_game->LoadLevelAsync().then([this]()
        {
            m_game->FinalizeLoadLevel();
            m_updateState = UpdateEngineState::ResourcesLoaded;
        }, task_continuation_context::use_current());
        ...
    ...
}
```

You can see a call to the `Simple3DGame::LoadLevelAsync` method (which returns a PPL `task<void>`). After that is a *continuation* that does some synchronous work.

`LoadLevelAsync` is asynchronous, but it doesn't return a value. So no value is being passed from the task to the continuation.

We can make the same kind of change to the code in these two places. The code is explained after the listing below. We could have a discussion here about the safe way to access the *this* pointer in a class-member coroutine. But let's defer that for a later section ([The deferred discussion about `co\_await` and the *this* pointer](#))—for now, this code works.

C++/CX

```
winrt::fire_and_forget GameMain::Update()
{
    ...
}
```

```

    case UpdateEngineState::WaitingForPress:
        ...
        co_await m_game->LoadLevelAsync();
        m_game->FinalizeLoadLevel();
        m_updateState = UpdateEngineState::ResourcesLoaded;
        ...
    case UpdateEngineState::Dynamics:
        ...
        co_await m_game->LoadLevelAsync();
        m_game->FinalizeLoadLevel();
        m_updateState = UpdateEngineState::ResourcesLoaded;
        ...
    ...
}

```

As you can see, because **LoadLevelAsync** returns a task, we can `co_await` it. And we don't need an explicit continuation—the code that follows a `co_await` executes only when **LoadLevelAsync** completes.

Introducing the `co_await` turns the method into a coroutine, so we couldn't leave it returning `void`. It's a fire-and-forget method, so we changed it to return [`winrt::fire\_and\_forget`](#).

You'll also need to edit `GameMain.h`. Change the return type of **GameMain::Update** from `void` to `winrt::fire_and_forget` in the declaration there, too.

You can make this change to your copy of the project, and the game still builds and runs the same. The source code is still fundamentally C++/CX, but it's now using the same patterns as C++/WinRT, so that has moved us a little closer to being able to port the rest of the code mechanically.

GameMain::ResetGame

GameMain::ResetGame is another fire-and-forget method; it calls **LoadLevelAsync**, too. So you can make the same code change there if you want the practice.

GameMain::OnDeviceRestored

Things get a little more interesting in **GameMain::OnDeviceRestored** because of its deeper nesting of async code, including a no-op task. Here's an outline of the asynchronous parts of the method (with the less-interesting synchronous code represented by ellipses).

```

void GameMain::OnDeviceRestored()
{
    ...
    create_task([this]()
    {
        return m_renderer->CreateGameDeviceResourcesAsync(m_game);
    }).then([this]()
    {
        ...
        if (m_updateState == UpdateEngineState::WaitingForResources)
        {
            ...
            return m_game->LoadLevelAsync().then([this]()
            {
                ...
            }, task_continuation_context::use_current());
        }
        else
        {
            return create_task([]()
            {
                // Return a no-op task.
            });
        }
    }, task_continuation_context::use_current()).then([this]()
    {
        ...
    }, task_continuation_context::use_current());
}

```

First, change the return type of **GameMain::OnDeviceRestored** from `void` to `wintr::fire_and_forget` in `GameMain.h` and `.cpp`. You'll also need to open `DeviceResources.h` and make the same change to the return type of **IDeviceNotify::OnDeviceRestored**.

To port the async code, remove all of the `create_task` and `then` calls and their curly brackets, and simplify the method into a flat series of statements.

Change any `return` that returns a task into a `co_await`. You'll be left with one `return` that returns nothing, so just delete that. When you're done, the no-op task will have disappeared, and the outline of the asynchronous parts of the method will look like this. Again, the less-interesting synchronous code is elided.

C++/CX

```

winrt::fire_and_forget GameMain::OnDeviceRestored()
{
    ...
    co_await m_renderer->CreateGameDeviceResourcesAsync(m_game);
}

```

```

...
if (m_updateState == UpdateEngineState::WaitingForResources)
{
    co_await m_game->LoadLevelAsync();
    ...
}
...
}

```

As you can see, this form of async structure is significantly simpler, and easier to read.

GameMain::GameMain

The **GameMain::GameMain** constructor performs work asynchronously, and no part of the project waits for that work to complete. Again, this listing outlines the asynchronous parts.

C++/CX

```

GameMain::GameMain(...) : ...
{
    ...
    create_task([this]()
    {
        ...
        return m_renderer->CreateGameDeviceResourcesAsync(m_game);
    }).then([this]()
    {
        ...
        if (m_updateState == UpdateEngineState::WaitingForResources)
        {
            return m_game->LoadLevelAsync().then([this]()
            {
                ...
            }, task_continuation_context::use_current());
        }
        else
        {
            return create_task([]()
            {
                // Return a no-op task.
            });
        }
    }, task_continuation_context::use_current()).then([this]()
    {
        ....
    }, task_continuation_context::use_current());
}

```

But a constructor can't return `winrt::fire_and_forget`, so we'll move the asynchronous code into a new `GameMain::ConstructInBackground` fire-and-forget method, flatten the code into `co_await` statements, and call the new method from the constructor. Here's the result.

C++/CX

```
GameMain::GameMain(...) : ...
{
    ...
    ConstructInBackground();
}

winrt::fire_and_forget GameMain::ConstructInBackground()
{
    ...
    co_await m_renderer->CreateGameDeviceResourcesAsync(m_game);
    ...
    if (m_updateState == UpdateEngineState::WaitingForResources)
    {
        ...
        co_await m_game->LoadLevelAsync();
        ...
    }
    ...
}
```

Now all of the fire-and-forget methods—in fact, all of the async code—in `GameMain` has been turned into coroutines. If you feel so inclined, perhaps you could look for fire-and-forget methods in other classes, and make similar changes.

The deferred discussion about `co_await` and the *this* pointer

When we were making changes to `GameMain::Update`, I deferred a discussion about the *this* pointer. Let's have that discussion here.

This applies to all of the methods we've changed so far; and it applies to *all* coroutines, not just fire-and-forget ones. Introducing a `co_await` into a method introduces a *suspension point*. And because of that, we have to be careful with the *this* pointer, which of course we make use of *after* the suspension point each time we access a class member.

The short story is that the solution is to call `implements::get_strong`. But for a complete discussion of the issue and the solution, see [Safely accessing the *this* pointer in a class-member coroutine](#).

You can call `implements::get_strong` only in a class that derives from [winrt::implements](#).

Derive GameMain from winrt::implements

The first change we need to make is in `GameMain.h`.

C++/CX

```
class GameMain :  
    public DX::IDeviceNotify
```

`GameMain` will continue to implement `DX::IDeviceNotify`, but we'll change it to derive from [winrt::implements](#).

C++/WinRT

```
class GameMain :  
    public winrt::implements<GameMain,  
        winrt::Windows::Foundation::IIinspectable>,  
        DX::IDeviceNotify
```

Next, in `App.cpp`, you'll find this method.

C++/CX

```
void App::Load(Platform::String^)  
{  
    if (!m_main)  
    {  
        m_main = std::unique_ptr<GameMain>(new GameMain(m_deviceResources));  
    }  
}
```

But now that `GameMain` derives from `winrt::implements`, we need to construct it in a different way. In this case, we'll use the [winrt::make_self](#) function template. For more info, see [Instantiating and returning implementation types and interfaces](#).

Replace that line of code with this.

C++/WinRT

```
...  
m_main = winrt::make_self<GameMain>(m_deviceResources);  
...
```


To close the loop on that change, we'll also need to change the type of *m_main*. In `App.h`, you'll find this code.

C++/CX

```
ref class App sealed :
    public Windows::ApplicationModel::Core::IFrameworkView
{
    ...
private:
    ...
    std::unique_ptr<GameMain> m_main;
};
```

Change that declaration of *m_main* to this.

C++/WinRT

```
...
winrt::com_ptr<GameMain> m_main;
...
```

We can now call `implements::get_strong`

For `GameMain::Update`, and for any of the other methods we added a `co_await` to, here's how you can call `get_strong` at the beginning of a coroutine to ensure that a strong reference survives until the coroutine completes.

C++/CX

```
winrt::fire_and_forget GameMain::Update()
{
    auto strong_this{ get_strong() }; // Keep *this* alive.
    ...
    co_await ...
    ...
}
```

Await `task<void>` within a `task<void>` method

The next simplest case is awaiting `task<void>` within a method that itself returns `task<void>`. That's because we can `co_await` a `task<void>`, and we can `co_return` from one.

You'll find a very simple example in the implementation of the method **Simple3DGame::LoadLevelAsync**. It's in the source code file `Simple3DGame.cpp`.

C++/CX

```
task<void> Simple3DGame::LoadLevelAsync()
{
    m_level[m_currentLevel]->Initialize(m_objects);
    m_levelDuration = m_level[m_currentLevel]->TimeLimit() +
m_levelBonusTime;
    return m_renderer->LoadLevelResourcesAsync();
}
```

There's just some synchronous code, followed by returning the task that's created by **GameRenderer::LoadLevelResourcesAsync**.

Instead of returning that task, we `co_await` it, and then `co_return` the resulting `void`.

C++/CX

```
task<void> Simple3DGame::LoadLevelAsync()
{
    m_level[m_currentLevel]->Initialize(m_objects);
    m_levelDuration = m_level[m_currentLevel]->TimeLimit() +
m_levelBonusTime;
    co_return co_await m_renderer->LoadLevelResourcesAsync();
}
```

That doesn't look like a profound change. But now that we're calling **GameRenderer::LoadLevelResourcesAsync** via `co_await`, we're free to port it to return a **wintrt::IAsyncXxx** instead of a task. We'll do that later in the section [Port a `task<void>` return type to `wintrt::IAsyncXxx`](#).

Await `task<void>` within a `task<T>` method

Although there are no suitable examples to be found in **Simple3DGameDX**, we can contrive a hypothetical example just to show the pattern.

The first line in the code example below demonstrates the simple `co_await` of a `task<void>`. Then, in order to satisfy the `task<T>` return type, we need to asynchronously return a **StorageFile**[^]. To do that, we `co_await` a Windows Runtime API, and `co_return` the resulting file.

C++/CX

```
task<StorageFile^> Simple3DGame::LoadLevelAndRetrieveFileAsync(
    StorageFolder^ location,
    Platform::String^ filename)
{
    co_await m_renderer->LoadLevelResourcesAsync();
    co_return co_await location->GetFileAsync(filename);
}
```

We could even port more of the method to C++/WinRT like this.

C++/CX

```
winrt::Windows::Foundation::IAsyncOperation<winrt::Windows::Storage::Storage
File>
Simple3DGame::LoadLevelAndRetrieveFileAsync(
    StorageFolder location,
    std::wstring filename)
{
    co_await m_renderer->LoadLevelResourcesAsync();
    co_return co_await location.GetFilesAsync(filename);
}
```

The *m_renderer* data member is still C++/CX in that example.

Await an `IAsyncXxx^` in a task method, leaving the rest of the project unchanged

We've seen how you can `co_await task<void>`. You can also `co_await` a method that returns an `IAsyncXxx`, whether that's a method in your project, or an asynchronous Windows API (for example, [StorageFolder.GetFilesAsync](#), which we cooperatively awaited in the previous section).

For an example of where we can make this kind of code change, let's look at **BasicReaderWriter::ReadDataAsync** (you'll find it implemented in `BasicReaderWriter.cpp`).

Here's the original C++/CX version.

C++/CX

```
task<Platform::Array<byte>^> BasicReaderWriter::ReadDataAsync(
    _In_ Platform::String^ filename
)
{
    return task<StorageFile^>(m_location->GetFileAsync(filename)).then([=]
    (StorageFile^ file)
```

```

{
    return FileIO::ReadBufferAsync(file);
}).then([=](IBuffer^ buffer)
{
    auto fileData = ref new Platform::Array<byte>(buffer->Length);
    DataReader::FromBuffer(buffer)->ReadBytes(fileData);
    return fileData;
});
}

```

The code listing below shows that we can `co_await` Windows APIs that return `IAsyncXxx^`. Not only that, we can also `co_return` the value that `BasicReaderWriter::ReadDataAsync` returns asynchronously (in this case, an array of bytes). This first step shows how to make just those changes; we'll actually port the C++/CX code to C++/WinRT in the next section.

C++/CX

```

task<Platform::Array<byte>^> BasicReaderWriter::ReadDataAsync(
    _In_ Platform::String^ filename
)
{
    StorageFile^ file = co_await m_location->GetFileAsync(filename);
    IBuffer^ buffer = co_await FileIO::ReadBufferAsync(file);
    auto fileData = ref new Platform::Array<byte>(buffer->Length);
    DataReader::FromBuffer(buffer)->ReadBytes(fileData);
    co_return fileData;
}

```

Again, we don't need to change the *callers* of the methods we're changing, because we didn't change the return type.

Port ReadDataAsync (mostly) to C++/WinRT, leaving the rest of the project unchanged

We can go a step further and port the method *almost entirely* to C++/WinRT without needing to change any other part of the project.

The only dependency that this method has on the rest of the project is the `BasicReaderWriter::m_location` data member, which is a C++/CX `StorageFolder^`. To leave that data member unchanged, and to leave the parameter type and return type unchanged, we need only perform a couple of conversions—one at the beginning of the method, and one at the end. For that, we can use the `from_cx` and `to_cx` interop helper functions.

Here's how `BasicReaderWriter::ReadDataAsync` looks after porting its implementation predominantly to C++/WinRT. This is a good example of *porting gradually*. And this method is at the stage where we can move away from thinking of it as *a C++/CX method that uses some C++/WinRT techniques*, and see it as *a C++/WinRT method that interoperates with C++/CX*.

C++/WinRT

```
#include <winrt/Windows.Storage.h>
#include <winrt/Windows.Storage.Streams.h>
#include <robuffer.h>
...
task<Platform::Array<byte>^> BasicReaderWriter::ReadDataAsync(
    _In_ Platform::String^ filename)
{
    auto location_from_cx = from_cx<winrt::Windows::Storage::StorageFolder>
(m_location);

    auto file = co_await location_from_cx.GetFilesAsync(filename->Data());
    auto buffer = co_await
winrt::Windows::Storage::FileIO::ReadBufferAsync(file);
    byte* bytes;
    auto byteAccess =
buffer.as<Windows::Storage::Streams::IBufferByteAccess>();
    winrt::check_hresult(byteAccess->Buffer(&bytes));

    co_return ref new Platform::Array<byte>(bytes, buffer.Length());
}
```

ⓘ Note

In `ReadDataAsync` above, we construct and return a new C++/CX array. And of course we do that to satisfy the method's return type (so that we don't have to change the rest of the project).

You may come across other examples in your own project where, after porting, you reach the end of the method and all you have is a C++/WinRT object. To `co_return` that, just call `to_cx` to convert it. There's more info about that, and an example, the next section.

Convert a `winrt::IAsyncXxx<T>` to a `task<T>`

This section deals with the situation where you've ported an asynchronous method to C++/WinRT (so that it returns a `winrt::IAsyncXxx<T>`), but you still have C++/CX code calling that method as if it's still returning a task.

- One case is where **T** is primitive, which needs no conversion.
- The other case is where **T** is a Windows Runtime type, in which case you'll need to convert that to a **T^**.

Convert a `winrt::IAsyncXxx<T>` (T is primitive) to a `task<T>`

The pattern in this section applies when you're asynchronously returning a primitive value (we'll use a Boolean value to illustrate). Consider an example where a method that you've already ported to C++/WinRT has this signature.

C++/WinRT

```
winrt::Windows::Foundation::IAsyncOperation<bool>
MyClass::GetBoolMemberFunctionAsync()
{
    bool value = ...
    co_return value;
}
```

You can convert a call to that method into a task like this.

C++/CX

```
task<bool> MyClass::RetrieveBoolTask()
{
    co_return co_await GetBoolMemberFunctionAsync();
}
```

Or like this.

C++/CX

```
task<bool> MyClass::RetrieveBoolTask()
{
    return concurrency::create_task(
        [this]() -> concurrency::task<bool> {
            auto result = co_await GetBoolMemberFunctionAsync();
            co_return result;
        });
}
```

Notice that the **task** return type of the lambda function is explicit, because the compiler can't deduce it.

We could also call the method from within an arbitrary task chain like this. Again, with an explicit lambda return type.

C++/CX

```
...
.then([this]() -> concurrency::task<bool> {
    co_return co_await GetBoolMemberFunctionAsync();
}).then([this](bool result) {
    ...
});
...
```

Convert a `winrt::IAsyncXxx<T>` (T is a Windows Runtime type) to a `task<T^>`

The pattern in this section applies when you're asynchronously returning a Windows Runtime value (we'll use a **StorageFile** value to illustrate). Consider an example where a method that you've already ported to C++/WinRT has this signature.

C++/WinRT

```
winrt::Windows::Foundation::IAsyncOperation<winrt::Windows::Storage::StorageFile>
MyClass::GetStorageFileMemberFunctionAsync()
{
    co_return co_await
winrt::Windows::Storage::StorageFile::GetFileFromPathAsync
    (L"MyFile.txt");
}
```

This next listing shows how to convert a call to that method into a task. Notice that we need to call the `to_cx` interop helper function to convert the returned C++/WinRT object into a C++/CX handle (also known as a *hat*) object.

C++/CX

```
task<Windows::Storage::StorageFile^> RetrieveStorageFileTask()
{
    winrt::Windows::Storage::StorageFile storageFile =
        co_await GetStorageFileMemberFunctionAsync();
    co_return to_cx<Windows::Storage::StorageFile>(storageFile);
}
```

Here's a more succinct version of that.

C++/CX

```
task<Windows::Storage::StorageFile^> RetrieveStorageFileTask()
{
    co_return to_cx<Windows::Storage::StorageFile>
(GetStorageFileMemberFunctionAsync());
}
```

And you can even choose to wrap that pattern up into a reusable function template, and `return` it just like you'd normally return a task.

C++/CX

```
template<typename ResultTypeCX, typename Awaitable>
concurrency::task<ResultTypeCX^> to_task(Awaitable awaitable)
{
    co_return to_cx<ResultTypeCX>(co_await awaitable);
}

task<Windows::Storage::StorageFile^> RetrieveStorageFileTask()
{
    return to_task<Windows::Storage::StorageFile>
(GetStorageFileMemberFunctionAsync());
}
```

If you like that idea, you might want to add `to_task` to `interop_helpers.h`.

Wrap `create_async` around a task that uses `co_return`

You can't `co_return` an `IAsyncXxx^` directly, but you can achieve something similar. If you have a task that cooperatively returns a value, then you can wrap that inside a call to `concurrency::create_async`.

Here's a hypothetical example, since there isn't an example we can lift from `Simple3DGameDX`.

C++/CX

```
Windows::Foundation::IAsyncOperation<bool>^ MyClass::RetrieveBoolAsync()
{
    return concurrency::create_async(
        [this]() -> concurrency::task<bool> {
            bool result = co_await GetBoolMemberFunctionAsync();
            co_return result;
        });
}
```



```
    });  
}
```

As you can see, you could obtain the return value from any method that you can `co_await`.

Port `concurrency::wait` to `co_await winrt::resume_after`

There are a couple of places where `Simple3DGameDX` uses `concurrency::wait` to pause the thread for a short amount of time. Here's an example.

C++/CX

```
// GameConstants.h  
namespace GameConstants  
{  
    ...  
    static const int InitialLoadingDelay = 2000;  
    ...  
}  
  
// GameRenderer.cpp  
task<void> GameRenderer::CreateGameDeviceResourcesAsync(_In_ Simple3DGame^  
game)  
{  
    std::vector<task<void>> tasks;  
    ...  
    tasks.push_back(create_task([]()  
    {  
        wait(GameConstants::InitialLoadingDelay);  
    }));  
    ...  
}
```

The C++/WinRT version of `concurrency::wait` is the `winrt::resume_after` struct. We can `co_await` that struct inside a PPL task. Here's a code example.

```
// GameConstants.h  
namespace GameConstants  
{  
    using namespace std::literals::chrono_literals;  
    ...  
    static const auto InitialLoadingDelay = 2000ms;  
    ...  
}
```

```
// GameRenderer.cpp
task<void> GameRenderer::CreateGameDeviceResourcesAsync(_In_ Simple3DGame^
game)
{
    std::vector<task<void>> tasks;
    ...
    tasks.push_back(create_task([]() -> task<void>
    {
        co_await winrt::resume_after(GameConstants::InitialLoadingDelay);
    }));
    ...
}
```

Notice the two other changes that we had to make. We changed the type of `GameConstants::InitialLoadingDelay` to `std::chrono::duration`, and we made the return type of the lambda function explicit, because the compiler is no longer able to deduce it.

Port a `task<void>` return type to `winrt::IAsyncXxx`

`Simple3DGame::LoadLevelAsync`

At this stage in our work with `Simple3DGameDX`, all of the places in the project that call `Simple3DGame::LoadLevelAsync` use `co_await` to call it.

That means that we can simply change that method's return type from `task<void>` to `winrt::Windows::Foundation::IAsyncAction` (leaving the rest of it unchanged).

C++/CX

```
winrt::Windows::Foundation::IAsyncAction Simple3DGame::LoadLevelAsync()
{
    m_level[m_currentLevel]->Initialize(m_objects);
    m_levelDuration = m_level[m_currentLevel]->TimeLimit() +
m_levelBonusTime;
    co_return co_await m_renderer->LoadLevelResourcesAsync();
}
```

It should now be fairly mechanical to port the rest of that method, and its dependencies (such as `m_level`, and so on), to C++/WinRT.

`GameRenderer::LoadLevelResourcesAsync`

Here's the original C++/CX version of `GameRenderer::LoadLevelResourcesAsync`.

C++/CX

```
// GameConstants.h
namespace GameConstants
{
    ...
    static const int LevelLoadingDelay = 500;
    ...
}

// GameRenderer.cpp
task<void> GameRenderer::LoadLevelResourcesAsync()
{
    m_levelResourcesLoaded = false;

    return create_task([this]()
    {
        wait(GameConstants::LevelLoadingDelay);
    });
}
```

`Simple3DGame::LoadLevelAsync` is the only place in the project that calls `GameRenderer::LoadLevelResourcesAsync`, and it already uses `co_await` to call it.

So there's no longer any need for `GameRenderer::LoadLevelResourcesAsync` to return a task—it can return a `wintr::Windows::Foundation::IAsyncAction` instead. And the implementation itself is simple enough to port completely to C++/WinRT. That involves making the same change we made in [Port concurrency::wait to co_await winrt::resume_after](#). And there are no significant dependencies on the rest of the project to worry about.

So here's how the method looks after porting it completely to C++/WinRT.

C++/WinRT

```
// GameConstants.h
namespace GameConstants
{
    using namespace std::literals::chrono_literals;
    ...
    static const auto LevelLoadingDelay = 500ms;
    ...
}

// GameRenderer.cpp
winrt::Windows::Foundation::IAsyncAction
GameRenderer::LoadLevelResourcesAsync()
{
```

```

    m_levelResourcesLoaded = false;
    co_return co_await
winrt::resume_after(GameConstants::LevelLoadingDelay);
}

```

The goal—fully port a method to C++/WinRT

Let's wrap up this walkthrough with an example of the end goal, by fully porting the method **BasicReaderWriter::ReadDataAsync** to C++/WinRT.

Last time we looked at this method (in the section [Port ReadDataAsync \(mostly\) to C++/WinRT, leaving the rest of the project unchanged](#)), it was *mostly* ported to C++/WinRT. But it still returned a task of **Platform::Array<byte>^**.

C++/WinRT

```

task<Platform::Array<byte>^> BasicReaderWriter::ReadDataAsync(
    _In_ Platform::String^ filename)
{
    auto location_from_cx = from_cx<winrt::Windows::Storage::StorageFolder>
(m_location);

    auto file = co_await location_from_cx.GetFilesAsync(filename->Data());
    auto buffer = co_await
winrt::Windows::Storage::FileIO::ReadBufferAsync(file);
    byte* bytes;
    auto byteAccess =
buffer.as<Windows::Storage::Streams::IBufferByteAccess>();
    winrt::check_hresult(byteAccess->Buffer(&bytes));

    co_return ref new Platform::Array<byte>(bytes, buffer.Length());
}

```

Instead of returning a task, we'll change it to return an **IAsyncOperation**. And instead of returning an array of bytes via that **IAsyncOperation**, we'll instead return a C++/WinRT **IBuffer** object. That will also require a minor change to the code at the call sites, as we'll see.

Here's how the method looks after porting its implementation, its parameter, and the *m_location* data member to use C++/WinRT syntax and objects.

C++/WinRT

```

winrt::Windows::Foundation::IAsyncOperation<winrt::Windows::Storage::Streams
::IBuffer>
BasicReaderWriter::ReadDataAsync(
    _In_ winrt::hstring const& filename)

```

```

{
    StorageFile file{ co_await m_location.GetFilesAsync(filename) };
    co_return co_await FileIO::ReadBufferAsync(file);
}

winrt::array_view<byte> BasicLoader::GetBufferView(
    winrt::Windows::Storage::Streams::IBuffer const& buffer)
{
    byte* bytes;
    auto byteAccess =
buffer.as<Windows::Storage::Streams::IBufferByteAccess>();
    winrt::check_hresult(byteAccess->Buffer(&bytes));
    return { bytes, bytes + buffer.Length() };
}

```

As you can see, **BasicReaderWriter::ReadDataAsync** itself is much simpler, because we've factored into its own method the synchronous logic that retrieves bytes from the buffer.

But now we need to port the call sites from this kind of structure in C++/CX.

C++/CX

```

task<void> BasicLoader::LoadTextureAsync(...)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then(
        [=](const Platform::Array<byte>^ textureData)
        {
            CreateTexture(...);
        });
}

```

To this pattern in C++/WinRT.

C++/WinRT

```

winrt::Windows::Foundation::IAsyncAction BasicLoader::LoadTextureAsync(...)
{
    auto textureBuffer = co_await
m_basicReaderWriter.ReadDataAsync(filename);
    auto textureData = GetBufferView(textureBuffer);
    CreateTexture(...);
}

```

Important APIs

- [IAsyncAction](#)
- [IAsyncActionWithProgress<TProgress>](#)

- [IAsyncOperation<TResult>](#)
- [IAsyncOperationWithProgress<TResult, TProgress>](#)
- [implements::get_strong](#)
- [concurrency::create_async](#)
- [concurrency::create_task](#)
- [concurrency::task](#)
- [concurrency::task::then](#)
- [concurrency::wait](#)
- [winrt::fire_and_forget](#)
- [winrt::make_self](#)

Related topics

- [Move to C++/WinRT from C++/CX](#)
- [Interop between C++/WinRT and C++/CX](#)
- [Concurrency and asynchronous operations with C++/WinRT](#)
- [Strong and weak references in C++/WinRT](#)
- [Author APIs with C++/WinRT](#)

Move to C++/WinRT from WRL

Article • 10/20/2022

This topic shows how to port [Windows Runtime C++ Template Library \(WRL\)](#) code to its equivalent in [C++/WinRT](#).

The first step in porting to C++/WinRT is to manually add C++/WinRT support to your project (see [Visual Studio support for C++/WinRT](#)). To do that, install the [Microsoft.Windows.CppWinRT NuGet package](#) into your project. Open the project in Visual Studio, click **Project > Manage NuGet Packages... > Browse**, type or paste **Microsoft.Windows.CppWinRT** in the search box, select the item in search results, and then click **Install** to install the package for that project. One effect of that change is that support for [C++/CX](#) is turned off in the project. If you're using C++/CX in the project, then you can leave support turned off and update your C++/CX code to C++/WinRT as well (see [Move to C++/WinRT from C++/CX](#)). Or you can turn support back on (in project properties, **C/C++ > General > Consume Windows Runtime Extension > Yes (/ZW)**), and first focus on porting your WRL code. C++/CX and C++/WinRT code can coexist in the same project, with the exception of XAML compiler support, and Windows Runtime components (see [Move to C++/WinRT from C++/CX](#)).

Set project property **General > Target Platform Version** to 10.0.17134.0 (Windows 10, version 1803) or greater.

In your precompiled header file (usually `pch.h`), include `winrt/base.h`.

C++/WinRT

```
#include <winrt/base.h>
```

If you include any C++/WinRT projected Windows API headers (for example, `winrt/Windows.Foundation.h`), then you don't need to explicitly include `winrt/base.h` like this because it will be included automatically for you.

Porting WRL COM smart pointers (Microsoft::WRL::ComPtr)

Port any code that uses `Microsoft::WRL::ComPtr<T>` to use `winrt::com_ptr<T>`. Here's a before-and-after code example. In the *after* version, the `com_ptr::put` member function retrieves the underlying raw pointer so that it can be set.

C++

```
ComPtr<IDXGIAdapter1> previousDefaultAdapter;  
DX::ThrowIfFailed(m_dxgiFactory->EnumAdapters1(0, &previousDefaultAdapter));
```

C++/WinRT

```
winrt::com_ptr<IDXGIAdapter1> previousDefaultAdapter;  
winrt::check_hresult(m_dxgiFactory->EnumAdapters1(0,  
previousDefaultAdapter.put()));
```

Important

If you have a `winrt::com_ptr` that's already seated (its internal raw pointer already has a target) and you want to re-seat it to point to a different object, then you first need to assign `nullptr` to it—as shown in the code example below. If you don't, then an already-seated `com_ptr` will draw the issue to your attention (when you call `com_ptr::put` or `com_ptr::put_void`) by asserting that its internal pointer is not null.

C++/WinRT

```
winrt::com_ptr<IDXGISwapChain1> m_pDXGISwapChain1;  
...  
// We execute the code below each time the window size changes.  
m_pDXGISwapChain1 = nullptr; // Important because we're about to re-seat  
winrt::check_hresult(  
    m_pDxgiFactory->CreateSwapChainForHwnd(  
        m_pCommandQueue.get(), // For Direct3D 12, this is a pointer to a  
        direct command queue, and not to the device.  
        m_hWnd,  
        &swapChainDesc,  
        nullptr,  
        nullptr,  
        m_pDXGISwapChain1.put()  
    );
```

In this next example (in the *after* version), the `com_ptr::put_void` member function retrieves the underlying raw pointer as a pointer to a pointer to void.

C++

```
ComPtr<ID3D12Debug> debugController;  
if (SUCCEEDED(D3D12GetDebugInterface(IID_PPV_ARGS(&debugController))))  
{  
    debugController->EnableDebugLayer();  
}
```


C++/WinRT

```
winrt::com_ptr<ID3D12Debug> debugController;  
if (SUCCEEDED(D3D12GetDebugInterface(__uuidof(debugController),  
debugController.put_void())))  
{  
    debugController->EnableDebugLayer();  
}
```

Replace **ComPtr::Get** with **com_ptr::get**.

C++

```
m_d3dDevice->CreateDepthStencilView(m_depthStencil.Get(), &dsvDesc,  
m_dsvHeap->GetCPUDescriptorHandleForHeapStart());
```

C++/WinRT

```
m_d3dDevice->CreateDepthStencilView(m_depthStencil.get(), &dsvDesc,  
m_dsvHeap->GetCPUDescriptorHandleForHeapStart());
```

When you want to pass the underlying raw pointer to a function that expects a pointer to **IUnknown**, use the **winrt::get_unknown** free function, as shown in this next example.

C++

```
ComPtr<IDXGISwapChain1> swapChain;  
DX::ThrowIfFailed(  
    m_dxgiFactory->CreateSwapChainForCoreWindow(  
        m_commandQueue.Get(),  
        reinterpret_cast<IUnknown*>(m_window.Get()),  
        &swapChainDesc,  
        nullptr,  
        &swapChain  
    )  
);
```

C++/WinRT

```
winrt::agile_ref<winrt::Windows::UI::Core::CoreWindow> m_window;  
winrt::com_ptr<IDXGISwapChain1> swapChain;  
winrt::check_hresult(  
    m_dxgiFactory->CreateSwapChainForCoreWindow(  
        m_commandQueue.get(),  
        winrt::get_unknown(m_window.get()),  
        &swapChainDesc,  
        nullptr,  
    )  
);
```

```
        swapChain.put()  
    )  
};
```

Porting a WRL module (Microsoft::WRL::Module)

This section relates to porting code that uses the [Microsoft::WRL::Module](#) type.

You can gradually add C++/WinRT code to an existing project that uses WRL to implement a component, and your existing WRL classes will continue to be supported. This section shows how.

If you create a new **Windows Runtime Component (C++/WinRT)** project type in Visual Studio, and build, then the file `Generated Files\module.g.cpp` is generated for you. That file contains the definitions of two useful C++/WinRT functions (listed out below), which you can copy and add to your project. Those function are **WINRT_CanUnloadNow** and **WINRT_GetActivationFactory** and, as you can see, they conditionally call WRL in order to support you whatever stage of porting you're at.

C++/WinRT

```
HRESULT WINRT_CALL WINRT_CanUnloadNow()  
{  
    #ifdef _WRL_MODULE_H_  
        if  
        (!::Microsoft::WRL::Module<::Microsoft::WRL::InProc>::GetModule().Terminate()  
        )  
        {  
            return S_FALSE;  
        }  
    #endif  
  
    if (winrt::get_module_lock())  
    {  
        return S_FALSE;  
    }  
  
    winrt::clear_factory_cache();  
    return S_OK;  
}  
  
HRESULT WINRT_CALL WINRT_GetActivationFactory(HSTRING classId, void**  
factory)  
{  
    try  
    {  
        *factory = nullptr;  
    }
```

```

        wchar_t const* const name = WINRT_WindowsGetStringRawBuffer(classId,
        nullptr);

        if (0 == wcscmp(name, L"MoveFromWRLTest.Class"))
        {
            *factory =
winrt::detach_abi(winrt::make<winrt::MoveFromWRLTest::factory_implementation
::Class>());
            return S_OK;
        }

#ifdef _WRL_MODULE_H_
        return
::Microsoft::WRL::Module<::Microsoft::WRL::InProc>::GetModule().GetActivatio
nFactory(classId, reinterpret_cast<::IActivationFactory*>(factory));
#else
        return winrt::hresult_class_not_available().to_abi();
#endif
    }
    catch (...) { return winrt::to_hresult(); }
}

```

Once you have these functions in your project, instead of calling [Module::GetActivationFactory](#) directly, call **WINRT_GetActivationFactory** (which calls the WRL function internally). Here's a before-and-after code example.

C++

```

HRESULT WINAPI DllGetActivationFactory(_In_ HSTRING activatableClassId,
_Out_ ::IActivationFactory **factory)
{
    auto & module =
Microsoft::WRL::Module<Microsoft::WRL::InProc>::GetModule();
    return module.GetActivationFactory(activatableClassId, factory);
}

```

C++/WinRT

```

HRESULT __stdcall WINRT_GetActivationFactory(HSTRING activatableClassId,
void** factory);
HRESULT WINAPI DllGetActivationFactory(_In_ HSTRING activatableClassId,
_Out_ ::IActivationFactory **factory)
{
    return WINRT_GetActivationFactory(activatableClassId,
reinterpret_cast<void*>(factory));
}

```

Instead of calling [Module::Terminate](#) directly, call **WINRT_CanUnloadNow** (which calls the WRL function internally). Here's a before-and-after code example.

C++

```
HRESULT __stdcall DllCanUnloadNow(void)
{
    auto &module =
Microsoft::WRL::Module<Microsoft::WRL::InProc>::GetModule();
    HRESULT hr = (module.Terminate() ? S_OK : S_FALSE);
    if (hr == S_OK)
    {
        hr = ...
    }
    return hr;
}
```

C++/WinRT

```
HRESULT __stdcall WINRT_CanUnloadNow();
HRESULT __stdcall DllCanUnloadNow(void)
{
    HRESULT hr = WINRT_CanUnloadNow();
    if (hr == S_OK)
    {
        hr = ...
    }
    return hr;
}
```

Porting Microsoft::WRL::Wrappers wrappers

This section relates to porting code that uses the [Microsoft::WRL::Wrappers](#) wrappers.

As you can see in the table below, to replace the threading helpers we recommend that you use the Standard C++ [thread support library](#)[↗]. A one-to-one mapping from the WRL wrappers could be misleading, since your choice depends on your needs. Also, some types that might seem obvious mappings are new to the C++20 standard, so those will be impractical if you haven't upgraded yet.

Type	Porting notes
CriticalSection class	Use the thread support library [↗]
Event class (WRL)	Use the winrt::event struct template
HandleT class	Use the winrt::handle struct or the winrt::file_handle struct
HString class	Use the winrt::hstring struct

Type	Porting notes
HStringReference class	No replacement, because C++/WinRT handles this internally in a way that's just as efficient as HStringReference with the advantage that you don't have to think about it.
Mutex class	Use the thread support library 
RoInitializeWrapper class	Use winrt::init_apartment and winrt::uninit_apartment ; or write your own trivial wrapper around CoInitializeEx and CoUninitialize .
Semaphore class	Use the thread support library 
SRWLock class	Use the thread support library 

Important APIs

- [winrt::com_ptr struct template](#)
- [winrt::Windows::Foundation::IUnknown struct](#)

Related topics

- [Introduction to C++/WinRT](#)
- [Move to C++/WinRT from C++/CX](#)
- [Windows Runtime C++ Template Library \(WRL\)](#)

Porting the Clipboard sample to C++/WinRT from C#—a case study

Article • 10/20/2022

This topic presents a case study of porting one of the [Universal Windows Platform \(UWP\) app samples](#) from C# to C++/WinRT. You can gain porting practice and experience by following along with the walkthrough and porting the sample for yourself as you go.

For a comprehensive catalog of the technical details involved in porting to C++/WinRT from C#, see the companion topic [Move to C++/WinRT from C#](#).

A brief preface about C# and C++ source code files

In a C# project, your source code files are primarily `.cs` files. When you move to C++, you'll notice that there are more kinds of source code files to get used to. The reason is to do with the difference between compilers, the way C++ source code is reused, and the notions of *declaring* and *defining* a type and its functions (its methods).

A function *declaration* describes just the function's *signature* (its return type, its name, and its parameter types and names). A function *definition* includes the function's *body* (its implementation).

It's a little different when it comes to types. You *define* a type by providing its name and by (at a minimum) just *declaring* all of its member functions (and other members). That's right, you can *define* a type even if you don't define its member functions.

- Common C++ source code files are `.h` (*dot aitch*) and `.cpp` files. A `.h` file is a *header* file, and it defines one or more types. While you *can* define member functions in a header, that's typically what a `.cpp` file is for. So for a hypothetical C++ type **MyClass**, you'd define **MyClass** in `MyClass.h`, and you'd define its member functions in `MyClass.cpp`. For other developers to re-use your classes, you'd share out just the `.h` files and object code. You'd keep your `.cpp` files secret, because the implementation constitutes your intellectual property.
- Precompiled header (`pch.h`). Typically there's a set of header files that you include in your application, and you don't change those files very often. So rather than processing the contents of that set of headers each time you compile, you can aggregate those headers into one file, compile that once, and then use the output

of that precompilation step each time you build. You do that via a *precompiled header* file (usually named `pch.h`).

- `.idl` files. These files contain Interface Definition Language (IDL). You can think of IDL as header files for Windows Runtime types. We'll talk more about IDL in the section [IDL for the MainPage type](#).

Download and test the Clipboard sample

Visit the [Clipboard sample](#) web page, and click **Download ZIP**. Unzip the downloaded file, and take a look at the folder structure.

- The C# version of the sample source code is contained in the folder named `cs`.
- The C++/WinRT version of the sample source code is contained in the folder named `cppwinrt`.
- Other files—used by both the C# version and the C++/WinRT version—can be found in the `shared` and `SharedContent` folders.

The walkthrough in this topic shows how you can recreate the C++/WinRT version of the Clipboard sample by porting it from the C# source code. That way, you can see how you can port your own C# projects to C++/WinRT.

To get a feel for what the sample does, open the C# solution (`\Clipboard_sample\cs\Clipboard.sln`), change the configuration as appropriate (perhaps to `x64`), build, and run. The sample's own user interface (UI) guides you through its various features, step by step.

Tip

The root folder of the sample that you downloaded might be named `Clipboard` rather than `Clipboard_sample`. But we'll continue to refer to that folder as `Clipboard_sample` to distinguish it from the C++/WinRT version that you'll be creating in a later step.

Create a Blank App (C++/WinRT), named Clipboard

Note

For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

Begin the porting process by creating a new C++/WinRT project in Microsoft Visual Studio. Create a new project using the **Blank App (C++/WinRT)** project template. Set its name to *Clipboard*, and (so that your folder structure will match the walkthrough) make sure that **Place solution and project in the same directory** is unchecked.

Just to get a baseline, make sure that this new, empty, project builds and runs.

Package.appxmanifest, and asset files

If the C# and C++/WinRT versions of the sample don't need to be installed side by side on the same machine, then the two projects' app package manifest source files (`Package.appxmanifest`) can be identical. In that case, you can just copy `Package.appxmanifest` from the C# project to the C++/WinRT project, and you're done.

For the two versions of the sample to coexist, they need different identifiers. In that case, in the C++/WinRT project, open the `Package.appxmanifest` file in an XML editor, and make a note of these three values.

- Inside the `/Package/Identity` element, note the value of the **Name** attribute. This is the *package name*. For a newly-created project, the project will give it an initial value of a unique GUID.
- Inside the `/Package/Applications/Application` element, note the value of the **Id** attribute. This is the *application id*.
- Inside the `/Package/mp:PhoneIdentity` element, note the value of the **PhoneProductId** attribute. Again, for a newly-created project, this will be set to the same GUID as the package name is set to.

Then copy `Package.appxmanifest` from the C# project to the C++/WinRT project. Finally, you can restore the three values that you noted. Or you can edit the copied values to make them unique and/or appropriate for the application and for your organization (as you ordinarily would for a new project). For example, in this case instead of restoring the value of the package name, we can just change the copied value from *Microsoft.SDKSamples.Clipboard.CS* to *Microsoft.SDKSamples.Clipboard.CppWinRT*. And we can leave the application id set to *App*. As long as either the package name or the application id are different, then the two applications will have different Application User Model IDs (AUMIDs). And that's what's necessary for two apps to be installed side by side on the same machine.

For the purposes of this walkthrough, it makes sense to make a few other changes in `Package.appxmanifest`. There are three occurrences of the string *Clipboard C# Sample*. Change that to *Clipboard C++/WinRT Sample*.

In the C++/WinRT project, the `Package.appxmanifest` file and the project are now out of sync with respect to the asset files that they reference. To remedy that, first remove the assets from the C++/WinRT project by selecting all the files in the `Assets` folder (in Solution Explorer in Visual Studio) and removing them (choose **Delete** in the dialog).

The C# project references asset files from a shared folder. You can do the same in the C++/WinRT project, or you can copy the files as we'll do in this walkthrough.

Navigate to the `\Clipboard_sample\SharedContent\media` folder. Select the seven files that the C# project includes (`microsoft-sdk.png`, `smalltile-sdk.png`, `splash-sdk.png`, `squaretile-sdk.png`, `storelogo-sdk.png`, `tile-sdk.png`, and `windows-sdk.png`), copy them, and paste them into the `\Clipboard\Clipboard\Assets` folder in the new project.

Right-click the `Assets` folder (in Solution Explorer in the C++/WinRT project) > **Add** > **Existing item...** and navigate to `\Clipboard\Clipboard\Assets`. In the file picker, select the seven files and click **Add**.

`Package.appxmanifest` is now back in sync with the project's asset files.

MainPage, including the functionality that configures the sample

The Clipboard sample—like all of the [Universal Windows Platform \(UWP\) app samples](#)—consists of a collection of scenarios that the user can step through one at a time. The collection of scenarios in a given sample is configured in the sample's source code. Each scenario in the collection is a data item that stores a title, as well as the type of the class in the project that implements the scenario.

In the C# version of the sample, if you look in the `SampleConfiguration.cs` source code file, you'll see two classes. Most of the configuration logic is in the **MainPage** class, which is a partial class (it forms a complete class when combined with the markup in `MainPage.xaml` and the imperative code in `MainPage.xaml.cs`). The other class in this source code file is **Scenario**, with its **Title** and **ClassType** properties.

Over the next few subsections, we'll look at how to port **MainPage** and **Scenario**.

IDL for the MainPage type

Let's begin this section by talking briefly about Interface Definition Language (IDL), and how it helps us when we're programming with C++/WinRT. IDL is a kind of source code that describes the callable surface of a Windows Runtime type. The callable (or public) surface of a type is *projected* out into the world, so that the type can be consumed. That *projected* portion of the type contrasts with the actual internal implementation of the type, which is of course not callable, and not public. It's only the projected portion that we define in IDL.

Having authored IDL source code (within an `.idl` file), you can then compile the IDL into machine-readable metadata files (also known as Windows Metadata). Those metadata files have the extension `.winmd`, and here are some of their uses.

- A `.winmd` can describe the Windows Runtime types in a component. When you reference a Windows Runtime Component (WRC) from an application project, the application project reads the Windows Metadata belonging to the WRC (that metadata may be in a separate file, or it may be packaged into the same file as the WRC itself) so that you can consume the WRC's types from within the application.
- A `.winmd` can describe the Windows Runtime types in one part of your application so that they can be consumed by a different part of the same application. For example, a Windows Runtime type that's consumed by a XAML page in the same app.
- To make it easier for you to consume Windows Runtime types (built-in or third party), the C++/WinRT build system uses `.winmd` files to generate wrapper types to represent the projected portions of those Windows Runtime types.
- To make it easier for you to implement your own Windows Runtime types, the C++/WinRT build system turns your IDL into a `.winmd` file, and then uses that to generate wrappers for your projection, as well as stubs on which to base your implementation (we'll talk more about these stubs later in this topic).

The specific version of IDL that we use with C++/WinRT is [Microsoft Interface Definition Language 3.0](#). In the remainder of this section of the topic, we'll examine the C# **MainPage** type in some detail. We'll decide which parts of it need to be in the *projection* of the C++/WinRT **MainPage** type (that is, in its callable, or public, surface), and which can be just part of its implementation. That distinction is important because when we come to author our IDL (which we'll do in the section after this one), we'll be defining only the callable parts in there.

The C# source code files that together implement the **MainPage** type are:

`MainPage.xaml` (which we'll port soon, by copying it), `MainPage.xaml.cs`, and `SampleConfiguration.cs`.

In the C++/WinRT version, we'll factor our **MainPage** type into source code files in a similar way. We'll take the logic in `MainPage.xaml.cs` and translate it for the most part to `MainPage.h` and `MainPage.cpp`. And for the logic in `SampleConfiguration.cs`, we'll translate that to `SampleConfiguration.h` and `SampleConfiguration.cpp`.

The classes in a C# Universal Windows Platform (UWP) application are of course Windows Runtime types. But when you author a type in a C++/WinRT application, you can choose whether that type is a Windows Runtime type, or a regular C++ class/struct/enumeration.

Any XAML page in our project needs to be a Windows Runtime type, so **MainPage** needs to be a Windows Runtime type. In the C++/WinRT project, **MainPage** is already a Windows Runtime type, so we don't need to change that aspect of it. Specifically, it's a *runtime class*.

- For more details about whether or not you should author a runtime class for a given type, see the topic [Author APIs with C++/WinRT](#).
- In C++/WinRT, the internal implementation of a runtime class, and the projected (public) parts of it, exist in the form of two different classes. These are known as the *implementation type* and the *projected type*. You can learn more about them in the topic mentioned in the previous bullet-point, and also in [Consume APIs with C++/WinRT](#).
- For more info about the connection between runtime classes and IDL (`.idl` files), you can read and follow along with the topic [XAML controls; bind to a C++/WinRT property](#). That topic walks through the process of authoring a new runtime class, the first step of which is to add a new **Midl File (.idl)** item to the project.

For **MainPage**, we actually have the necessary `MainPage.idl` file already in the C++/WinRT project. That's because the project template created it for us. But later in this walkthrough we'll be adding further `.idl` files to the project.

We'll shortly see a listing of exactly what IDL we need to add to the existing `MainPage.idl` file. Before that, we have some reasoning to do about what does, and what doesn't, need to go in the IDL.

To determine which members of **MainPage** we need to declare in `MainPage.idl` (so that they become part of the **MainPage** runtime class), and which can simply be members of the **MainPage** implementation type, let's make a list of the members of the C# **MainPage** class. We find those members by looking in `MainPage.xaml.cs` and in `SampleConfiguration.cs`.

We find a total of twelve `protected` and `private` fields and methods. And we find the following `public` members.

- The default constructor `MainPage()`.
- The static fields `Current` and `FEATURE_NAME`.
- The properties `IsClipboardContentChangedEnabled` and `Scenarios`.
- The methods `BuildClipboardFormatsOutputString`, `DisplayToast`, `EnableClipboardContentChangedNotifications`, and `NotifyUser`.

It's those `public` members that are candidates for declaring in `MainPage.idl`. So let's examine each one and see whether they need to be part of the `MainPage` runtime class, or whether they need only to be part of its implementation.

- The default constructor `MainPage()`. For a XAML **Page**, it's normal to declare a default constructor in its IDL. That way, the XAML UI framework can activate the type.
- The static field `Current` is used from within the individual scenario XAML pages to access the application's instance of `MainPage`. Since `Current` isn't being used to interoperate with the XAML framework (nor is it used across compilation units), we could reserve it to be solely a member of the implementation type. With your own projects in cases like this, you might choose to do that. But since the field is an instance of the projected type, it feels logical to declare it in the IDL. So that's what we'll do here (and doing so also makes the code slightly cleaner).
- It's a similar case for the static `FEATURE_NAME` field, which is accessed within the `MainPage` type. Again, choosing to declare it in the IDL makes our code slightly cleaner.
- The property `IsClipboardContentChangedEnabled` is used only in the `OtherScenarios` class. So during the port, we'll simplify things a little, and make it a private field of the `OtherScenarios` runtime class. So that one won't go in the IDL.
- The property `Scenarios` is a collection of objects of type `Scenario` (a type that we mentioned earlier). We'll talk about `Scenario` in the next subsection, so let's leave the `Scenarios` property until then, too.
- The methods `BuildClipboardFormatsOutputString`, `DisplayToast`, and `EnableClipboardContentChangedNotifications` are utility functions that feel more to do with the general state of the sample than about the main page. So during the port, we'll refactor these three methods onto a new utility type named `SampleState` (which doesn't need to be a Windows Runtime type). For that reason, these three methods won't go in the IDL.
- The method `NotifyUser` is called from within the individual scenario XAML pages on the instance of `MainPage` that's returned from the static `Current` field. Since (as already noted) `Current` is an instance of the projected type, we need to declare

NotifyUser in the IDL. **NotifyUser** takes a parameter of type **NotifyType**. We'll talk about that in the next subsection.

Any member that you want to databind to also needs to be declared in IDL (whether you're using `{x:Bind}` or `{Binding}`). For more info, see [Data binding](#).

We're making progress: we're developing a list of which members to add, and which not to add, to the `MainPage.idl` file. But we still have to discuss the **Scenarios** property, and the **NotifyType** type. So let's do that next.

IDL for the Scenario and NotifyType types

The **Scenario** class is defined in `SampleConfiguration.cs`. We have a decision to make about how to port that class to C++/WinRT. By default, we would probably make it an ordinary C++ `struct`. But if **Scenario** is being used across binaries, or to interoperate with the XAML framework, then it needs to be declared in IDL as a Windows Runtime type.

Studying the C# source code, we find that **Scenario** is used in this context.

XAML

```
<ListBox x:Name="ScenarioControl" ... >
```

C#

```
var itemCollection = new List<Scenario>();
int i = 1;
foreach (Scenario s in scenarios)
{
    itemCollection.Add(new Scenario { Title = $"{i++}) {s.Title}", ClassType
    = s.ClassType });
}
ScenarioControl.ItemsSource = itemCollection;
```

A collection of **Scenario** objects is being assigned to the [ItemsSource](#) property of a **ListBox** (which is an items control). Since **Scenario** *does* need to interoperate with XAML, it needs to be a Windows Runtime type. So it needs to be defined in IDL. Defining the **Scenario** type in IDL causes the C++/WinRT build system to generate a source code definition of **Scenario** for you in a behind-the-scenes header file (the name and location of which are not important for this walkthrough).

And you'll recall that **MainPage.Scenarios** is a collection of **Scenario** objects, which we've just said need to be in IDL. For that reason, **MainPage.Scenarios** itself also needs

to be declared in the IDL.

NotifyType is an `enum` declared in C#'s `MainPage.xaml.cs`. Because we pass **NotifyType** to a method belonging to the **MainPage** runtime class, **NotifyType** too needs to be a Windows Runtime type; and it needs to be defined in `MainPage.idl`.

Now let's add to the `MainPage.idl` file the new types and the new member of **Mainpage** that we've decided to declare in IDL. At the same time, we'll remove from the IDL the placeholder members of **Mainpage** that the Visual Studio project template gave us.

So, in your C++/WinRT project, open `MainPage.idl`, and edit it so that it looks like the listing below. Note that one of the edits is to change the namespace name from **Clipboard** to **SDKTemplate**. If you like, you can just replace the entire contents of `MainPage.idl` with the following code. Another tweak to note is that we're changing the name of **Scenario::ClassType** to **Scenario::ClassName**.

idl

```
// MainPage.idl
namespace SDKTemplate
{
    struct Scenario
    {
        String Title;
        Windows.UI.Xaml.Interop.TypeName ClassName;
    };

    enum NotifyType
    {
        StatusMessage,
        ErrorMessage
    };

    [default_interface]
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();

        static MainPage Current{ get; };
        static String FEATURE_NAME{ get; };

        static Windows.Foundation.Collections.IVector<Scenario> scenarios{
get; };

        void NotifyUser(String strMessage, NotifyType type);
    };
}
```

❗ Note

For more info about the contents of an `.idl` file in a C++/WinRT project, see [Microsoft Interface Definition Language 3.0](#).

With your own porting work, you might not want or need to change the namespace name like we did above. We're doing it here only because the default namespace of the C# project that we're porting is **SDKTemplate**; while the name of the project and of the assembly is **Clipboard**.

But, as we proceed with the port in this walkthrough, we'll be changing every occurrence in source code of the **Clipboard** namespace name to **SDKTemplate**. There's also a place in C++/WinRT project properties where the **Clipboard** namespace name appears, so we'll take the opportunity to change that now.

In Visual Studio, for the C++/WinRT project, set the project property **Common Properties > C++/WinRT > Root Namespace** to the value *SDKTemplate*.

Save the IDL, and re-generate stub files

The topic [XAML controls; bind to a C++/WinRT property](#) introduces the notion of *stub files*, and shows you a walkthrough of them in action. We also mentioned stubs earlier in this topic when we mentioned that C++/WinRT build system turns the contents of your `.idl` files into Windows Metadata, and then from that metadata a tool named `cppwinrt.exe` generates stubs on which you can base your implementation.

Each time you add, remove, or change something in your IDL, and build, the build system updates the stub implementations in those stubs files. So each time you change your IDL and build, we recommend that you view those stubs files, copy any changed signatures, and paste them into your project. We'll give more specifics and examples of exactly how to do that in a moment. But the advantage of doing this is to give you an error-free way of knowing at all times what the shape of your implementation type should be, and what the signature of its methods should be.

At this point in the walkthrough, we're done editing the `MainPage.idl` file for the time being, so you should save it now. The project won't build to completion at the moment, but performing a build now is a useful thing to do because it regenerates the stub files for **MainPage**. So build the project now, and disregard any build errors.

For this C++/WinRT project, the stub files are generated in the `\Clipboard\Clipboard\Generated Files\sources` folder. You'll find them there after the

partial build has come to an end (again, as expected, the build won't succeed entirely. But the step that we're interested in—generating stubs—*will* have succeeded). The files we're interested in are `MainPage.h` and `MainPage.cpp`.

In those two stub files, you'll see new stub implementations of the members of **MainPage** that we added to the IDL (**Current** and **FEATURE_NAME**, for example). You'll want to copy those stub implementations into the `MainPage.h` and `MainPage.cpp` files that are already in the project. At the same time, just as we did with the IDL, we'll remove from those existing files the placeholder members of **Mainpage** that the Visual Studio project template gave us (the dummy property named **MyProperty**, and the event handler named **ClickHandler**).

In fact, the only member of the current version of **MainPage** that we want to keep is the constructor.

Once you've copied the new members from the stub files, deleted the members we don't want, and updated the namespace, the `MainPage.h` and `MainPage.cpp` files in your project should look like the code listings below. Notice that there are two **MainPage** types. One in the **implementation** namespace, and a second one in the **factory_implementation** namespace. The only change we've made to the **factory_implementation** one is to add **SDKTemplate** to its namespace.

C++/WinRT

```
// MainPage.h
#pragma once
#include "MainPage.g.h"

namespace winrt::SDKTemplate::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        MainPage();

        static SDKTemplate::MainPage Current();
        static hstring FEATURE_NAME();
        static
        Windows::Foundation::Collections::IVector<SDKTemplate::Scenario>
        scenarios();
        void NotifyUser(hstring const& strMessage, SDKTemplate::NotifyType
        const& type);
    };
}
namespace winrt::SDKTemplate::factory_implementation
{
    struct MainPage : MainPageT<MainPage, implementation::MainPage>
    {
```



```
};  
}
```

C++/WinRT

```
// MainPage.cpp  
#include "pch.h"  
#include "MainPage.h"  
#include "MainPage.g.cpp"  
  
namespace winrt::SDKTemplate::implementation  
{  
    MainPage::MainPage()  
    {  
        InitializeComponent();  
    }  
    SDKTemplate::MainPage MainPage::Current()  
    {  
        throw hresult_not_implemented();  
    }  
    hstring MainPage::FEATURE_NAME()  
    {  
        throw hresult_not_implemented();  
    }  
    Windows::Foundation::Collections::IVector<SDKTemplate::Scenario>  
MainPage::scenarios()  
    {  
        throw hresult_not_implemented();  
    }  
    void MainPage::NotifyUser(hstring const& strMessage,  
SDKTemplate::NotifyType const& type)  
    {  
        throw hresult_not_implemented();  
    }  
}
```

For strings, C# uses **System.String**. See the **MainPage.NotifyUser** method for an example. In our IDL, we declare a string with **String**, and when the `cppwinrt.exe` tool generates C++/WinRT code for us, it uses the **winrt::hstring** type. Any time we come across a string in C# code, we'll port that to **winrt::hstring**. For more info, see [String handling in C++/WinRT](#).

For an explanation of the `const&` parameters in the method signatures, see [Parameter-passing](#).

Update all remaining namespace declarations/references, and build

Before building the C++/WinRT project, find any declarations of (and references to) the **Clipboard** namespace, and change them to **SDKTemplate**.

- `MainPage.xaml` and `App.xaml`. The namespace appears in the values of the `x:Class` and `xmlns:local` attributes.
- `App.idl`.
- `App.h`.
- `App.cpp`. There are two `using namespace` directives (search for the substring `using namespace Clipboard`), and two qualifications of the **MainPage** type (search for `Clipboard::MainPage`). Those need changing.

Since we removed the event handler from **MainPage**, also go into `MainPage.xaml` and delete the **Button** element from the markup.

Save all the files. Clean the solution (**Build > Clean Solution**), and then build it. Having followed all of the changes so far, exactly as written, the build is expected to succeed.

Implement the MainPage members that we declared in IDL

The constructor, Current, and FEATURE_NAME

Here's the relevant code (from the C# project) that we need to port.

XAML

```
<!-- MainPage.xaml -->
...
<TextBlock x:Name="SampleTitle" ... />
...
```

C#

```
// MainPage.xaml.cs
...
public sealed partial class MainPage : Page
{
    public static MainPage Current;

    public MainPage()
    {
        InitializeComponent();
        Current = this;
        SampleTitle.Text = FEATURE_NAME;
    }
}
```

```

...
}
...

// SampleConfiguration.cs
...
public partial class MainPage : Page
{
    public const string FEATURE_NAME = "Clipboard C# sample";
    ...
}
...

```

Soon, we'll be re-using `MainPage.xaml` in its entirety (by copying it). For now (below), we'll temporarily add a **TextBlock** element, with the appropriate name, into the `MainPage.xaml` of the C++/WinRT project.

FEATURE_NAME is a static field of **MainPage** (a C# `const` field is essentially static in its behavior), defined in `SampleConfiguration.cs`. For C++/WinRT, instead of a (static) field, we'll make it the C++/WinRT expression of a (static) read-only property. The C++/WinRT way of expressing a property getter is as a function that returns the property value, and takes no parameters (an accessor). So the C# **FEATURE_NAME** static field becomes the C++/WinRT **FEATURE_NAME** static accessor function (in this case, returning the string literal).

Incidentally, we'd do the same thing if we were porting a C# read-only property. For a C# writeable property, the C++/WinRT way of expressing a property setter is as a `void` function that takes the property value as a parameter (a mutator). In either case, if the C# field or property is static, then so is the C++/WinRT accessor and/or mutator.

Current is a static (not a constant) field of **MainPage**. Again, we'll make it (the C++/WinRT expression of) a read-only property, and again make it static. Where **FEATURE_NAME** is constant, **Current** is not. So in C++/WinRT we'll need a backing field, and our accessor will return that. So, in the C++/WinRT project, we'll declare in `MainPage.h` a private static field named **current**, we'll define/initialize **current** in `MainPage.cpp` (because it has static storage duration), and we'll access it via a public static accessor function named **Current**.

The constructor itself performs a couple of assignments, which are straightforward to port.

In the C++/WinRT project, add a new **Visual C++ > Code > C++ File (.cpp)** item with the name `SampleConfiguration.cpp`.

Edit `MainPage.xaml`, `MainPage.h`, `MainPage.cpp`, and `SampleConfiguration.cpp` to match the listings below.

XAML

```
<!-- MainPage.xaml -->
...
<StackPanel ...>
    <TextBlock x:Name="SampleTitle" />
</StackPanel>
...
```

C++/WinRT

```
// MainPage.h
...
namespace winrt::SDKTemplate::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        ...
        static SDKTemplate::MainPage Current() { return current; }
        ...
    private:
        static SDKTemplate::MainPage current;
        ...
    };
    ...
}

// MainPage.cpp
...
namespace winrt::SDKTemplate::implementation
{
    SDKTemplate::MainPage MainPage::current{ nullptr };

    MainPage::MainPage()
    {
        InitializeComponent();
        MainPage::current = *this;
        SampleTitle().Text(FEATURE_NAME());
    }
    ...
}

// SampleConfiguration.cpp
#include "pch.h"
#include "MainPage.h"

using namespace winrt;
using namespace SDKTemplate;
```

```
hstring implementation::MainPage::FEATURE_NAME()
{
    return L"Clipboard C++/WinRT Sample";
}
```

Also, be sure to delete the existing function bodies from `MainPage.cpp` for `MainPage::Current()` and `MainPage::FEATURE_NAME()`, because we're now defining those methods elsewhere.

As you can see, `MainPage::current` is declared as being of type `SDKTemplate::MainPage`, which is the projected type. It's not of type `SDKTemplate::implementation::MainPage`, which is the implementation type. The projected type is the one that's designed to be consumed either within the project for XAML interoperation, or across binaries. The implementation type is what you use to implement the facilities that you've exposed on your projected type. Because the declaration of `MainPage::current` (in `MainPage.h`) appears within the implementation namespace (`wirt::SDKTemplate::implementation`), an unqualified `MainPage` would have referred to the implementation type. So, we qualify with `SDKTemplate::` in order to be clear that we want `MainPage::current` to be an instance of the projected type `wirt::SDKTemplate::MainPage`.

In the constructor, there are some points related to `MainPage::current = *this;` that deserve an explanation.

- When you use the `this` pointer inside a member of the implementation type, the `this` pointer is of course *a pointer to the implementation type*.
- To convert the `this` pointer to the corresponding projected type, dereference it. Provided you generate your implementation type from IDL (as we have here), the implementation type has a conversion operator that converts to its projected type. That's why the assignment here works.

For more info about those details, see [Instantiating and returning implementation types and interfaces](#).

Also in the constructor is `SampleTitle().Text(FEATURE_NAME());`. The `SampleTitle()` part is a call to a simple accessor function named `SampleTitle`, which returns the `TextBlock` that we added to the XAML. Whenever you `x:Name` a XAML element, the XAML compiler generates an accessor for you that's named for the element. The `.Text(...)` part calls the `Text` mutator function on the `TextBlock` object that the `SampleTitle` accessor returned. And `FEATURE_NAME()` calls our static `MainPage::FEATURE_NAME` accessor function to return the string literal. Altogether, that line of code sets the `Text` property of the `TextBlock` named `SampleTitle`.

Note that since strings are wide in the Windows Runtime, to port a string literal we prefix it with the wide-char encoding prefix `L`. So we change (for example) "a string literal" to `L"a string literal"`. Also see [Wide string literals](#).

Scenarios

Here's the relevant C# code that we need to port.

```
C#

// MainPage.xaml.cs
...
public sealed partial class MainPage : Page
{
    ...
    public List<Scenario> Scenarios
    {
        get { return this.scenarios; }
    }
    ...
}
...

// SampleConfiguration.cs
...
public partial class MainPage : Page
{
    ...
    List<Scenario> scenarios = new List<Scenario>
    {
        new Scenario() { Title = "Copy and paste text", ClassType =
typeof(CopyText) },
        new Scenario() { Title = "Copy and paste an image", ClassType =
typeof(CopyImage) },
        new Scenario() { Title = "Copy and paste files", ClassType =
typeof(CopyFiles) },
        new Scenario() { Title = "Other Clipboard operations", ClassType =
typeof(OtherScenarios) }
    };
    ...
}
...
```

From our earlier investigation, we know that this collection of **Scenario** objects is being displayed in a **ListBox**. In C++/WinRT, there are limits to *the kind of collection* that we can assign to the **ItemsSource** property of an items control. The collection must be either a vector or an observable vector, and its elements must be one of the following:

- Either runtime classes, or

- [IInspectable](#).

For the **IInspectable** case, if the elements are not themselves runtime classes, then those elements need to be of a kind that can be boxed and unboxed to and from [IInspectable](#). And that means that they have to be Windows Runtime types (see [Boxing and unboxing values to IInspectable](#)).

For this case study, we didn't make **Scenario** a runtime class. That is still a reasonable option, though. And there'll be cases in your own porting work where a runtime class will definitely be the way to go. For example, if you need to make the element type *observable* (see [XAML controls; bind to a C++/WinRT property](#)), or if the element needs to have methods for any other reason, and it's more than just a set of data members.

Since, in this walkthrough, we're *not* going with a runtime class for the **Scenario** type, then we need to think about boxing. If we'd made **Scenario** a regular C++ `struct`, then we wouldn't be able to box it. But we declared **Scenario** as a `struct` in IDL, and so we *can* box it.

We're left with the choice of boxing the **Scenario** ahead of time, or waiting until we're about to assign to the **ItemsSource**, and box them on a just-in-time basis. Here are some considerations regarding those two options.

- Boxing ahead of time. For this option, our data member is a collection of **IInspectable** ready to assign to the UI. On initialization, we box the **Scenario** objects into that data member. We need only one copy of that collection, but we have to unbox an element every time we needed to read its fields.
- Boxing just in time. For this option, our data member is a collection of **Scenario**. When the time comes to assign to the UI, we box the **Scenario** objects from the data member into a new collection of **IInspectable**. We can read the fields of the elements in the data member without unboxing, but we need two copies of the collection.

As you can see, for a small collection like this, the pros and cons make it something of a wash. So, for this case study, we'll go with the just-in-time option.

The **scenarios** member is a field of **MainPage**, defined and initialized in `SampleConfiguration.cs`. And **Scenarios** is a read-only property of **MainPage**, defined in `MainPage.xaml.cs` (and implemented to simply return the **scenarios** field). We'll do something similar in the C++/WinRT project; but we'll make the two members static (since we need only one instance across the application; and so that we can access them without needing a class instance). And we'll name them *scenariosInner* and *scenarios*, respectively. We'll declare *scenariosInner* in `MainPage.h`. And, because it has static

storage duration, we'll define/initialize it in a `.cpp` file (`SampleConfiguration.cpp`, in this case).

Edit `MainPage.h` and `SampleConfiguration.cpp` to match the listings below.

C++/WinRT

```
// MainPage.h
...
struct MainPage : MainPageT<MainPage>
{
    ...
    static Windows::Foundation::Collections::IVector<Scenario> scenarios() {
return scenariosInner; }
    ...
private:
    static winrt::Windows::Foundation::Collections::IVector<Scenario>
scenariosInner;
    ...
};

// SampleConfiguration.cpp
...
using namespace Windows::Foundation::Collections;
...
IVector<Scenario> implementation::MainPage::scenariosInner =
winrt::single_threaded_observable_vector<Scenario>(
{
    Scenario{ L"Copy and paste text", xaml_typename<SDKTemplate::CopyText>()
},
    Scenario{ L"Copy and paste an image",
xaml_typename<SDKTemplate::CopyImage>() },
    Scenario{ L"Copy and paste files", xaml_typename<SDKTemplate::CopyFiles>
() },
    Scenario{ L"History and roaming",
xaml_typename<SDKTemplate::HistoryAndRoaming>() },
    Scenario{ L"Other Clipboard operations",
xaml_typename<SDKTemplate::OtherScenarios>() },
});
```

Also, be sure to delete the existing function body from `MainPage.cpp` for `MainPage::scenarios()`, because we're now defining that method in the header file.

As you can see, in `SampleConfiguration.cpp`, we initialize the static data member `scenariosInner` by calling a C++/WinRT helper function named `winrt::single_threaded_observable_vector`. That function creates a new Windows Runtime collection object for us, and returns it as an **IObservableVector** interface. Since, in this sample, the collection is not *observable* (it doesn't need to be, because it doesn't add nor remove elements after initialization), we could instead have opted to call

`wint::single_threaded_vector`. That function returns the collection as an **IVector** interface.

For more info about collections, and binding to them, see [XAML items controls; bind to a C++/WinRT collection](#), and [Collections with C++/WinRT](#).

The initialization code you just added references types that aren't yet in the project (for example, `wint::SDKTemplate::CopyText`). To remedy that, let's go ahead and add five new blank XAML pages to the project.

Add five new blank XAML pages

Add a new **Visual C++ > Blank Page (C++/WinRT)** item to the project (be certain that it's the **Blank Page (C++/WinRT)** item template, and not the **Blank Page** one).

Name it `CopyText`. The new XAML page is defined within the **SDKTemplate** namespace, which is what we want.

Repeat the above process another four times, and named the XAML pages `CopyImage`, `CopyFiles`, `HistoryAndRoaming`, and `OtherScenarios`.

You'll now be able to build again, if you wish.

NotifyUser

In the C# project, you'll find the implementation of the **MainPage.NotifyUser** method in `MainPage.xaml.cs`. **MainPage.NotifyUser** has a dependency on **MainPage.UpdateStatus**, and that method in turn has dependencies on XAML elements that we haven't yet ported. So for now we'll just stub out an **UpdateStatus** method in the C++/WinRT project, and we'll port that later.

Here's the relevant C# code that we need to port.

C#

```
// MainPage.xaml.cs
...
public void NotifyUser(string strMessage, NotifyType type)
if (Dispatcher.HasThreadAccess)
{
    UpdateStatus(strMessage, type);
}
else
{
    var task = Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
UpdateStatus(strMessage, type));
}
```

```
private void UpdateStatus(string strMessage, NotifyType type) { ... }{  
...
```

NotifyUser uses the [Windows.UI.Core.CoreDispatcherPriority](#) enum. In C++/WinRT, whenever you want to use a type from a Windows namespace, you need to include the corresponding C++/WinRT Windows namespace header file (for more info about that, see [Get started with C++/WinRT](#)). In this case, as you'll see in the code listing below, the header is `winrt/Windows.UI.Core.h`, and we'll be including it in `pch.h`.

UpdateStatus is private. So we'll make that a private method on our **MainPage** implementation type. **UpdateStatus** isn't meant to be called on the runtime class, so we won't declare it in IDL.

After porting **MainPage.NotifyUser**, and stubbing out **MainPage.UpdateStatus**, this is what we have in the C++/WinRT project. After this code listing, we'll examine some of the details.

C++/WinRT

```
// pch.h  
...  
#include <winrt/Windows.UI.Core.h>  
...  
  
// MainPage.h  
...  
struct MainPage : MainPageT<MainPage>  
{  
...  
    void NotifyUser(hstring const& strMessage, SDKTemplate::NotifyType  
const& type);  
...  
private:  
    void UpdateStatus(hstring const& strMessage, SDKTemplate::NotifyType  
const& type);  
...  
};  
  
// MainPage.cpp  
...  
void MainPage::NotifyUser(hstring const& strMessage, SDKTemplate::NotifyType  
const& type)  
{  
    if (Dispatcher().HasThreadAccess())  
    {  
        UpdateStatus(strMessage, type);  
    }  
    else  
    {
```

```

Dispatcher().RunAsync(Windows::UI::Core::CoreDispatcherPriority::Normal,
[strMessage, type, this]()
{
    UpdateStatus(strMessage, type);
});
}
}
void MainPage::UpdateStatus(hstring const& strMessage,
SDKTemplate::NotifyType const& type)
{
    throw hresult_not_implemented();
}
...

```

In C#, you can use dot notation to *dot into* nested properties. So, the C# **MainPage** type can access its own **Dispatcher** property with the syntax `Dispatcher`. And C# can further *dot into* that value with syntax such as `Dispatcher.HasThreadAccess`. In C++/WinRT, properties are implemented as accessor functions, so the syntax differs only in that you add parentheses for each function call.

 Expand table

C#	C++/WinRT
<code>Dispatcher.HasThreadAccess</code>	<code>Dispatcher().HasThreadAccess()</code>

When the C# version of **NotifyUser** calls `CoreDispatcher.RunAsync`, it implements the asynchronous callback delegate as a lambda function. The C++/WinRT version does the same thing, but the syntax is a little different. In C++/WinRT, we *capture* the two parameters that we're going to use, as well as the `this` pointer (since we're going to call a member function). There's more info about implementing delegates as lambdas, and code examples, in the topic [Handle events by using delegates in C++/WinRT](#). Also, we can disregard the `var task =` part in this particular case. We're not waiting on the returned asynchronous object, so there's no need to store it.

Implement the remaining MainPage members

Let's make a full list of the members of **MainPage** (implemented across `MainPage.xaml.cs` and `SampleConfiguration.cs`) so that we can see which ones we've ported so far, and which ones are yet to do.

 Expand table

Member	Access	Status
MainPage constructor	public	Ported
Current property	public	Ported
FEATURE_NAME property	public	Ported
IsClipboardContentChangedEnabled property	public	Not started
Scenarios property	public	Ported
BuildClipboardFormatsOutputString method	public	Not started
DisplayToast method	public	Not started
EnableClipboardContentChangedNotifications method	public	Not started
NotifyUser method	public	Ported
OnNavigatedTo method	protected	Not started
isApplicationWindowActive field	private	Not started
needToPrintClipboardFormat field	private	Not started
scenarios field	private	Ported
Button_Click method	private	Not started
DisplayChangedFormats method	private	Not started
Footer_Click method	private	Not started
HandleClipboardChanged method	private	Not started
OnClipboardChanged method	private	Not started
OnWindowActivated method	private	Not started
ScenarioControl_SelectionChanged method	private	Not started
UpdateStatus method	private	Stubbed out

We'll talk about the as-yet-unported members in the next few subsections, then.

ⓘ Note

From time to time, we'll come across references in the source code to UI elements in the XAML markup (in `MainPage.xaml`). As we come to these references, we'll

temporarily work around them by adding simple placeholder elements to the XAML. That way, the project will continue to build after each subsection. The alternative is to resolve the references by copying the *entire* contents of `MainPage.xaml` from the C# project to the C++/WinRT project now. But if we do that then it'll be a long time before we can come to a pit stop and build again (thus potentially obscuring any typos or other errors that we make along the way).

Once we're done porting the imperative code for the **MainPage** class, *then* we'll copy the contents of the XAML file and be confident that the project will still build.

IsClipboardContentChangedEnabled

This is a get-set C# property that defaults to `false`. It's a member of **MainPage**, and is defined in `SampleConfiguration.cs`.

For C++/WinRT, we'll need an accessor function, a mutator function, and a backing data member as a field. Since **IsClipboardContentChangedEnabled** represents the state of one of the scenarios in the sample, rather than the state of **MainPage** itself, we'll create the new members on a new utility type called **SampleState**. And we'll implement that in our `SampleConfiguration.cpp` source code file, and we'll make the members `static` (since we need only one instance across the application; and so that we can access them without needing a class instance).

To accompany our `SampleConfiguration.cpp` in the C++/WinRT project, add a new **Visual C++ > Code > Header File (.h)** item with the name `SampleConfiguration.h`. Edit `SampleConfiguration.h` and `SampleConfiguration.cpp` to match the listings below.

C++/WinRT

```
// SampleConfiguration.h
#pragma once
#include "pch.h"

namespace winrt::SDKTemplate
{
    struct SampleState
    {
        static bool IsClipboardContentChangedEnabled();
        static void IsClipboardContentChangedEnabled(bool checked);
    private:
        static bool isClipboardContentChangedEnabled;
    };
}

// SampleConfiguration.cpp
```